

Public repository and reproducibility links. The complete public repository is github.com/splittersplatter/MAS. Direct links: [final report](#), [Problem 1 code](#), [Problem 1 animation](#), [Problem 2 code](#), and [Problem 3 code](#).

Problem 1

Fix the number of agents to be $N = 20$. Generate a connected graph using Erdos-Renyi model for N agents and some $p \in (0, 1)$. Assume random initial positions for all N agents in $\mathbb{R}^2$. Using formation control strategy, display your name in capital letters in $\mathbb{R}^2$, letter by letter in a sequence. For instance, my name is Venkat and this is how I would do:

- Starting from initial position vector $x[0]$, agents should go to positions forming the letter V.
- Starting from positions corresponding to letter V, agents should go to positions corresponding to letter E.
- Continue this letter sequence, until agents go to letter T.

Record your name formation sequence as an animation video through your simulation, host it in your github and simply give just the Github link containing both the code and the video as your solution. Ensure that your Github repository is made public before submitting. For people having big lengthy names, choose the corresponding name which people use to call you on a regular basis - not the full entire name. For instance, if a student's full name is Manoj Kumar Sahu, and he goes commonly by simply Manoj, then he should simulate the sequence MANOJ.

Solution: I use the name **DHRUV**, which is the short name used regularly. There are $N = 20$ agents, unit edge weights, and a connected $G(20, 0.25)$ graph. The graph was generated with seed 20260905; the first connected graph was used, so no edges were added by hand.

Notation note. The question calls the initial stacked position $x[0]$. In the solution I use $p_i[k]$ for the position of agent i and $p[k]$ for the stacked position, so the question's $x[0]$ is the same as $p[0]$ here.

For agent i , let $p_i[k] \in \mathbb{R}^2$ be its position and let $r_i^{(\ell)} \in \mathbb{R}^2$ be its target point for letter ℓ . I apply consensus to the translated positions:

$$z_i[k] = p_i[k] - r_i^{(\ell)}, \quad u_i[k] = \sum_{j \in \mathcal{N}_i} \left((p_j[k] - p_i[k]) - (r_j^{(\ell)} - r_i^{(\ell)}) \right).$$

With $p[k]$ and $r^{(\ell)}$ stacked in $\mathbb{R}^{40}$, the update is

$$p[k+1] = p[k] - \Delta t (L \otimes I_2) (p[k] - r^{(\ell)}), \quad L = D - A.$$

In simple terms: subtracting the desired points makes ordinary consensus produce the required relative shape.

Each capital letter is drawn from line segments in the same $[-2.5, 2.5] \times [-3, 3]$ frame. Exactly 20 points are sampled by arc length and the mean point is subtracted, so every letter has the same centre. The initial positions are sampled uniformly from $[-5, 5] \times [-4, 4]$. The final positions of one letter are used as the initial positions for the next letter.

The resulting formation-offset matrices are listed below. Entries are rounded to three decimals; the simulation uses the full-precision values returned by the code.

$$\begin{aligned}
 R^{(D)} &= \begin{bmatrix} -1.489 & -2.841 \\ -1.489 & -1.914 \\ -1.489 & -0.986 \\ -1.489 & -0.059 \\ -1.489 & 0.869 \\ -1.489 & 1.797 \\ -1.489 & 2.724 \\ -0.996 & 3.159 \\ -0.068 & 3.159 \\ 0.813 & 2.986 \\ 1.619 & 2.525 \\ 1.911 & 1.767 \\ 1.911 & 0.839 \\ 1.911 & -0.088 \\ 1.911 & -1.016 \\ 1.911 & -1.943 \\ 1.190 & -2.453 \\ 0.366 & -2.841 \\ -0.562 & -2.841 \\ -1.489 & -2.841 \end{bmatrix} & R^{(H)} &= \begin{bmatrix} -2.185 & -2.700 \\ -2.185 & -1.950 \\ -2.185 & -1.200 \\ -2.185 & -0.450 \\ -2.185 & 0.300 \\ -2.185 & 1.050 \\ -2.185 & 1.800 \\ -2.185 & 2.550 \\ 2.415 & -2.700 \\ 2.415 & -1.843 \\ 2.415 & -0.986 \\ 2.415 & -0.129 \\ 2.415 & 0.729 \\ 2.415 & 1.586 \\ 2.415 & 2.443 \\ -2.185 & 0.300 \\ -1.035 & 0.300 \\ 0.115 & 0.300 \\ 1.265 & 0.300 \\ 2.415 & 0.300 \end{bmatrix} & R^{(R)} &= \begin{bmatrix} -1.507 & -3.341 \\ -1.507 & -2.459 \\ -1.507 & -1.576 \\ -1.507 & -0.694 \\ -1.507 & 0.189 \\ -1.507 & 1.071 \\ -1.507 & 1.953 \\ -1.330 & 2.659 \\ -0.447 & 2.659 \\ 0.435 & 2.659 \\ 1.173 & 2.224 \\ 1.740 & 1.666 \\ 1.303 & 0.900 \\ 0.865 & 0.134 \\ 0.258 & -0.341 \\ -0.624 & -0.341 \\ 0.593 & -0.341 \\ 1.393 & -1.341 \\ 2.193 & -2.341 \\ 2.993 & -3.341 \end{bmatrix} \\
R^{(U)} &= \begin{bmatrix} -2.300 & 3.465 \\ -2.300 & 2.649 \\ -2.300 & 1.834 \\ -2.300 & 1.019 \\ -2.300 & 0.203 \\ -2.300 & -0.612 \\ -2.300 & -1.427 \\ -1.827 & -2.061 \\ -1.223 & -2.535 \\ -0.408 & -2.535 \\ 0.408 & -2.535 \\ 1.223 & -2.535 \\ 1.827 & -2.061 \\ 2.300 & -1.427 \\ 2.300 & -0.612 \\ 2.300 & 0.203 \\ 2.300 & 1.019 \\ 2.300 & 1.834 \\ 2.300 & 2.649 \\ 2.300 & 3.465 \end{bmatrix} & R^{(V)} &= \begin{bmatrix} -2.400 & 2.842 \\ -2.147 & 2.211 \\ -1.895 & 1.579 \\ -1.642 & 0.947 \\ -1.389 & 0.316 \\ -1.137 & -0.316 \\ -0.884 & -0.947 \\ -0.632 & -1.579 \\ -0.379 & -2.211 \\ -0.126 & -2.842 \\ 0.126 & -2.842 \\ 0.379 & -2.211 \\ 0.632 & -1.579 \\ 0.884 & -0.947 \\ 1.137 & -0.316 \\ 1.389 & 0.316 \\ 1.642 & 0.947 \\ 1.895 & 1.579 \\ 2.147 & 2.211 \\ 2.400 & 2.842 \end{bmatrix}
\end{aligned}$$

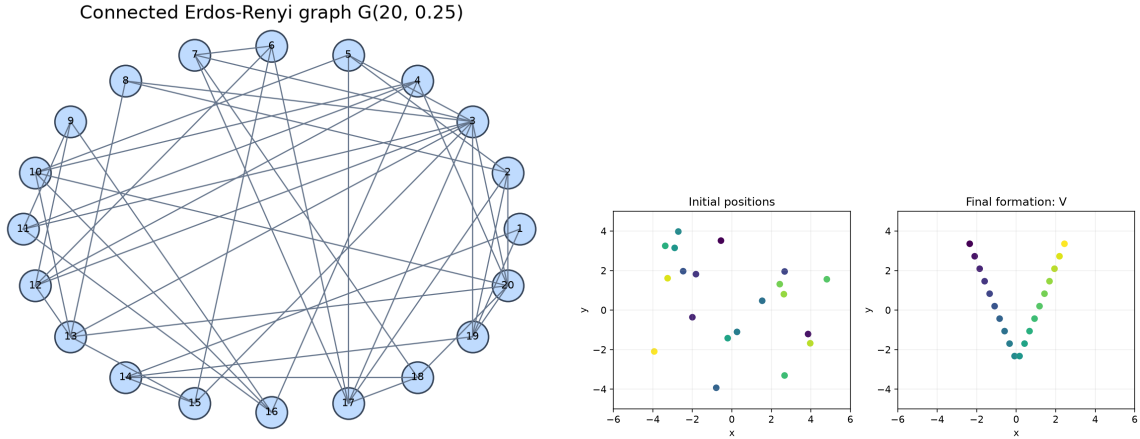


Figure 1: The connected communication graph, the random initial cloud, and the final V formation.

The graph has $\lambda_2(L) = 0.910671 > 0$, so it has exactly one zero Laplacian eigenvalue and is connected. The largest Laplacian eigenvalue is $\lambda_{\max}(L) = 11.496026$. I use $\Delta t = 0.12$, which

satisfies $0 < 0.12 < 2/\lambda_{\max} = 0.173973$. The stopping value is

$$\varepsilon_{\text{form}}[k] = \max_{(i,j) \in E} \left\| (p_i[k] - p_j[k]) - (r_i^{(\ell)} - r_j^{(\ell)}) \right\|,$$

with tolerance 10^{-3} . The final edge errors for D, H, R, U, and V are respectively 9.41×10^{-4} , 9.87×10^{-4} , 9.93×10^{-4} , 9.32×10^{-4} , and 9.43×10^{-4} .

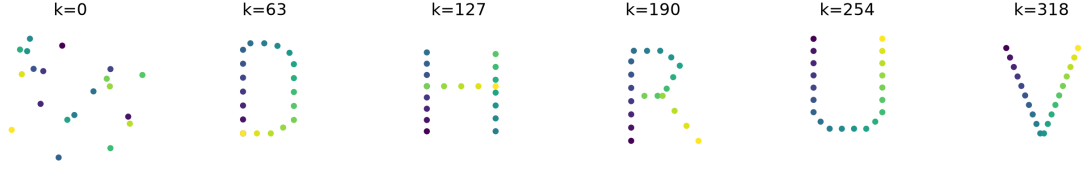


Figure 2: Selected fixed-axis frames from the DHRUV formation sequence.

The animation file is `assets/p1_DHRUV_formation.gif`, and the source code is included below. The code appendix is included because the submission rules require code in the PDF. The required public GitHub repository containing both the code and the animation is:

Public GitHub repository (Problem 1):
github.com/splittersplatter/MAS
Direct animation link: [p1_DHRUV_formation.gif](#)

Graph matrices

Rows and columns use the order $1, 2, \dots, 20$. The adjacency matrix has $a_{ij} = 1$ exactly when agents i and j communicate, and $a_{ii} = 0$.

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}.$$

$$D = \text{diag}(2, 5, 10, 5, 4, 4, 4, 3, 3, 5, 4, 5, 6, 4, 4, 4, 6, 4, 5, 7), \quad L = D - A.$$

This explicitly defines every entry of L and is less error-prone to read than repeating the same 20 by 20 array. The code below also prints the full numerical L matrix.

Problem 1 code

The exact source file is also available at [GitHub: Assignment_3_Problem_1.py](#).

```
"""Problem 1: reproducible formation-control animation for the name DHRUV."""
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import PillowWriter

ROOT = Path(__file__).resolve().parent
ASSET = ROOT / "assets"
ASSET.mkdir(exist_ok=True)
SEED, N, P = 20260905, 20, 0.25
DT, TOL, MAX_STEPS = 0.12, 1e-3, 260

def connected_graph():
    for s in range(SEED, SEED + 200):
        rng = np.random.default_rng(s)
        upper = rng.random((N, N)) < P
        A = np.triu(upper, 1).astype(float); A += A.T
```

```

        seen = {0}; frontier = [0]
        while frontier:
            i = frontier.pop()
            for j in np.flatnonzero(A[i]):
                if int(j) not in seen: seen.add(int(j)); frontier.append(int(j))
        if len(seen) == N: return A, s
    raise RuntimeError("No connected graph found in the documented retries.")

def sample_polyline(points, count, include_end=True):
    points = np.asarray(points, dtype=float)
    lengths = np.linalg.norm(np.diff(points, axis=0), axis=1)
    dist = np.r_[0.0, np.cumsum(lengths)]
    values = np.linspace(0.0, dist[-1], count, endpoint=include_end)
    out = []
    for d in values:
        j = min(np.searchsorted(dist, d, side="right") - 1, len(points) - 2)
        frac = 0.0 if lengths[j] == 0 else (d - dist[j]) / lengths[j]
        out.append(points[j] + frac * (points[j + 1] - points[j]))
    return np.asarray(out)

def letter_strokes(letter):
    h = 6.0
    if letter == "D":
        return [((-2.4, -h / 2), (-2.4, h / 2), (-0.4, h / 2), (1.0, 2.2),
                (1.0, -2.2), (-0.4, -h / 2), (-2.4, -h / 2))]
    if letter == "H":
        return [((-2.3, -h / 2), (-2.3, h / 2)], [(2.3, -h / 2), (2.3, h / 2)],
                [(-2.3, 0), (2.3, 0)])
    if letter == "R":
        return [((-2.3, -h / 2), (-2.3, h / 2), (-0.2, h / 2), (1.0, 2.1),
                (-0.2, 0), (-2.3, 0)], [(-0.2, 0), (2.2, -h / 2)])
    if letter == "U":
        return [((-2.3, h / 2), (-2.3, -2.0), (-1.4, -h / 2), (1.4, -h / 2),
                (2.3, -2.0), (2.3, h / 2))]
    if letter == "V":
        return [((-2.4, h / 2), (0, -h / 2), (2.4, h / 2))]
    raise ValueError(letter)

def offsets(letter):
    strokes = letter_strokes(letter)
    lengths = [np.linalg.norm(np.diff(np.asarray(s), axis=0), axis=1).sum() for s in strokes]
    counts = np.maximum(2, np.floor(N * np.asarray(lengths) / sum(lengths)).astype(int))
    while counts.sum() < N:
        counts[np.argmax(lengths)] += 1
    while counts.sum() > N:
        j = int(np.argmax(counts))
        if counts[j] > 2:
            counts[j] -= 1
        else:
            break
    chunks = [sample_polyline(s, int(c), include_end=(k == len(strokes) - 1))
              for k, (s, c) in enumerate(zip(strokes, counts))]
    r = np.vstack(chunks)
    return r - r.mean(axis=0)

def main():
    A, graph_seed = connected_graph(); edges = list(zip(*np.where(np.triu(A, 1) > 0)))
    L = np.diag(A.sum(axis=1)) - A
    lam_max = np.linalg.eigvalsh(L).max()
    rng = np.random.default_rng(SEED)
    p = rng.uniform([-5.0, -4.0], [5.0, 4.0], size=(N, 2))
    letters = "DHRUV"
    R = {letter: offsets(letter) for letter in letters}
    frames, records = [p.copy()], []
    for letter in letters:
        r = R[letter]

```

```

    eps = float("inf")
    for k in range(MAX_STEPS):
        eps = max((np.linalg.norm((p[i] - p[j]) - (r[i] - r[j])))
                  for i, j in edges), default=0.0)
        if eps <= TOL:
            break
        p = p - DT * (L @ (p - r))
        frames.append(p.copy())
    records.append((letter, k + 1, eps))
    frames.append(p.copy())

# Communication graph.
fig, ax = plt.subplots(figsize=(5.3, 4.2))
theta = np.linspace(0, 2 * np.pi, N, endpoint=False); pos = np.c_[np.cos(theta), np.sin(theta)]
for i, j in edges: ax.plot([pos[i,0],pos[j,0]],[pos[i,1],pos[j,1]], color="#64748b", lw=.8)
ax.scatter(pos[:,0], pos[:,1], s=430, c="#bfd8fe", edgecolors="#334155")
for i in range(N): ax.text(pos[i,0], pos[i,1], str(i+1), ha="center", va="center", fontsize=7)
ax.set_title(f"Connected Erdos-Renyi graph G({N}, {P})")
ax.axis("off")
fig.tight_layout(); fig.savefig(ASSET / "p1_graph.png", dpi=220); plt.close(fig)

# Initial cloud and final V shape.
fig, axes = plt.subplots(1, 2, figsize=(8.5, 3.8))
axes[0].scatter(frames[0][:, 0], frames[0][:, 1], c=np.arange(N), cmap="viridis")
axes[0].set_title("Initial positions"); axes[0].set_aspect("equal")
axes[1].scatter(p[:, 0], p[:, 1], c=np.arange(N), cmap="viridis")
axes[1].set_title("Final formation: V"); axes[1].set_aspect("equal")
for ax in axes:
    ax.set_xlim(-6, 6); ax.set_ylim(-5, 5); ax.set_xlabel("x"); ax.set_ylabel("y"); ax.grid(alpha=.2)
fig.tight_layout(); fig.savefig(ASSET / "p1_initial_final.png", dpi=220); plt.close(fig)

# A small frame strip for the report.
chosen = np.linspace(0, len(frames) - 1, 6, dtype=int)
fig, axes = plt.subplots(1, 6, figsize=(12, 2.4))
for ax, idx in zip(axes, chosen):
    xy = frames[idx]; ax.scatter(xy[:, 0], xy[:, 1], s=12, c=np.arange(N), cmap="viridis")
    ax.set_xlim(-6, 6); ax.set_ylim(-5, 5); ax.set_aspect("equal"); ax.set_title(f"k={idx}"); ax.axis(
        "off")
fig.tight_layout(); fig.savefig(ASSET / "p1_frame_strip.png", dpi=220); plt.close(fig)

# The full animation uses fixed axes and equal aspect ratio.
fig, ax = plt.subplots(figsize=(5.0, 4.0))
scat = ax.scatter([], [], c=[], cmap="viridis", s=30)
ax.set_xlim(-6, 6); ax.set_ylim(-5, 5); ax.set_aspect("equal"); ax.set_xlabel("x"); ax.set_ylabel("y")
ax.grid(alpha=.2)
def update(xy):
    scat.set_offsets(xy); return (scat,)
from matplotlib.animation import FuncAnimation
anim = FuncAnimation(fig, update, frames=frames, interval=35, blit=True)
anim.save(ASSET / "p1_DHRUV_formation.gif", writer=PillowWriter(fps=18)); plt.close(fig)

print(f"seed={SEED}, graph_seed={graph_seed}, N={N}, p={P}, edges={len(edges)}")
print(f"connected=True, lambda_max={lam_max:.6f}, dt={DT}, 2/lambda_max={2/lam_max:.6f}")
print("letters=", letters, "initial_rectangle=[-5,5] x [-4,4]")
for letter, steps, eps in records: print(f"{letter}: steps={steps}, final_edge_error={eps:.6e}")
print("A="); print(A.astype(int))
print("D="); print(np.diag(A.sum(axis=1)).astype(int))
print("L="); print(L.astype(int))

if __name__ == "__main__":
    main()

```

Problem 2

Suppose that there is a target intruder which has to be tracked by a swarm of $N \in \mathbb{N}$ drones. Each of the N drones are equipped with a sensor and the drones are connected by a communication network $G(V, E)$. The initial location of the intruder is denoted by $z_0 \in \mathbb{R}^3$ is not known exactly and $z_0 \sim \mathcal{N}(\bar{z}_0, \Sigma_0)$, where mean $\bar{z}_0 \in \mathbb{R}^3$ and covariance $\Sigma_0 \in \mathbb{R}^{3 \times 3}$ and is positive semi-definite.

The intruder target is doing a random walk with dynamics given by

$$z(t+1) = z(t) + w, \quad \forall t \in \mathbb{N},$$

where $w \sim \mathcal{N}(0, \Sigma_w)$ denotes the process noise (like the wind disturbance). Each sensor i is located at $x_i \in \mathbb{R}^3$ and assume that sensor i mounted on drone i knows its location x_i , and its measurement data $y_i(t)$ is given by the model $\forall t \in \mathbb{N}$ as

$$y_i(t) = x_i - z(t) + v_i.$$

Here v_i is a random sensor measurement noise with $v_i \sim \mathcal{N}(0, \Sigma_v^{(i)})$, where $\Sigma_v^{(i)} \succeq 0$ is also known to agent i . We assume that noises for different sensors are independent. Now let each drone with its respective sensor $i = 1, \dots, N$ take one measurement using the model and obtain $y_i(t)$ for each time $t = [0, T_{\max}]$. Do the following:

1. Formulate the distributed state estimation problem of the intruder's location $z(t)$ as a consensus optimization problem, and specify each local objective function f_i .
2. Find the estimate of the intruder's state $\hat{z}(t)$ using distributed state estimation technique (distributed gradient descent or distributed gradient tracking) with the data collected by all the sensors $\{y_i(t), t \in [0, T_{\max}]\}_{i=1}^N$ via the measurement model.
3. Assume and fix the number of drones $N \in \mathbb{N}$ (typically less than 20) and also generate a random graph $G(V, E)$ with N agents (using Erdos-Renyi model) containing a spanning tree (so that the graph is connected) and plot it. Assume and fix the random quantities for $\bar{z}_0, \Sigma_0, \Sigma_w, \{x_i, \Sigma_v^{(i)}\}_{i=1}^N$ depicting above figure. Simulate and plot the fixed sensor locations and actual true intruder location trajectory using the random-walk model at every time for $t = [0, T_{\max}]$. Plot the computed estimate of the intruder's state trajectory $\hat{z}(t)$ for all $t = [0, T_{\max}]$ and check if your estimate matches well with the actual intruder state $z(t)$ and report the estimation error $e(t) = \hat{z}(t) - z(t)$ too.

Solution: I use $N = 6$ drones and $T_{\max} = 30$. The graph is $G(6, 0.55)$ with seed 20260905 and is connected. The target model is written with separate noise samples:

$$\begin{aligned} z[0] &\sim \mathcal{N}(\bar{z}_0, \Sigma_0), \quad z[t+1] = z[t] + w[t], \quad w[t] \sim \mathcal{N}(0, \Sigma_w), \\ y_i[t] &= x_i - z[t] + v_i[t], \quad v_i[t] \sim \mathcal{N}(0, \Sigma_v^{(i)}). \end{aligned}$$

Because the sensor position is known, I first form

$$q_i[t] = x_i - y_i[t] = z[t] - v_i[t].$$

This sign change makes each local measurement a direct noisy copy of the target.

The question permits positive-semidefinite covariance matrices. The inverse-weighted objective used below needs invertible weights, so I choose positive-definite matrices in the simulation.

If a singular covariance were used, its inverse would be replaced by a stated regularised inverse or Moore–Penrose pseudoinverse.

2.1 Consensus optimization model

Let $\xi[t] \in \mathbb{R}^3$ be a candidate target position and stack the whole trajectory as $\xi = (\xi[0]^\top, \dots, \xi[T_{\max}]^\top)^\top \in \mathbb{R}^{3(T_{\max}+1)}$. Let $Q_0 = \Sigma_0^{-1}$, $Q_w = \Sigma_w^{-1}$, and $Q_i = (\Sigma_v^{(i)})^{-1}$. All chosen covariance matrices are positive definite, so these inverses exist. The local objective at drone i is

$$f_i(\xi) = \frac{1}{N} \left[(\xi[0] - \bar{z}_0)^\top Q_0 (\xi[0] - \bar{z}_0) + \sum_{t=0}^{T_{\max}-1} (\xi[t+1] - \xi[t])^\top Q_w (\xi[t+1] - \xi[t]) \right] + \sum_{t=0}^{T_{\max}} (q_i[t] - \xi[t])^\top Q_i (q_i[t] - \xi[t]).$$

The distributed problem is

$$\min_{\xi_1, \dots, \xi_N} \sum_{i=1}^N f_i(\xi_i) \quad \text{subject to} \quad \xi_i = \xi_j, \quad (i, j) \in E.$$

Equivalently, for $\Xi = [\xi_1^\top \ \dots \ \xi_N^\top]^\top$,

$$(L \otimes I_{3(T_{\max}+1)})\Xi = 0.$$

The prior and process terms are divided by N because they are common information, while each measurement term belongs to one sensor.

2.2 Gradient blocks and distributed update

For clarity, the gradient blocks are shown before stacking. For $t = 0$,

$$\nabla_{\xi[0]} f_i = \frac{2}{N} Q_0 (\xi[0] - \bar{z}_0) - \frac{2}{N} Q_w (\xi[1] - \xi[0]) + 2Q_i (\xi[0] - q_i[0]).$$

For an interior time $1 \leq t \leq T_{\max} - 1$,

$$\nabla_{\xi[t]} f_i = \frac{2}{N} Q_w (2\xi[t] - \xi[t-1] - \xi[t+1]) + 2Q_i (\xi[t] - q_i[t]).$$

For the final time,

$$\nabla_{\xi[T_{\max}]} f_i = \frac{2}{N} Q_w (\xi[T_{\max}] - \xi[T_{\max} - 1]) + 2Q_i (\xi[T_{\max}] - q_i[T_{\max}]).$$

Only neighbouring time blocks appear, so the Hessian is block tridiagonal.

I use distributed gradient tracking. Let W be symmetric and doubly stochastic. With $s_i^0 = \nabla f_i(\xi_i^0)$, the updates are

$$\xi_i^{k+1} = \sum_j W_{ij} \xi_j^k - \alpha_k s_i^k, \quad s_i^{k+1} = \sum_j W_{ij} s_j^k + \nabla f_i(\xi_i^{k+1}) - \nabla f_i(\xi_i^k).$$

The implementation uses $\alpha_k = 0.018/(1 + 0.0004k)$ and stops when the maximum edge disagreement is below 2×10^{-4} and the objective change is below 10^{-6} .

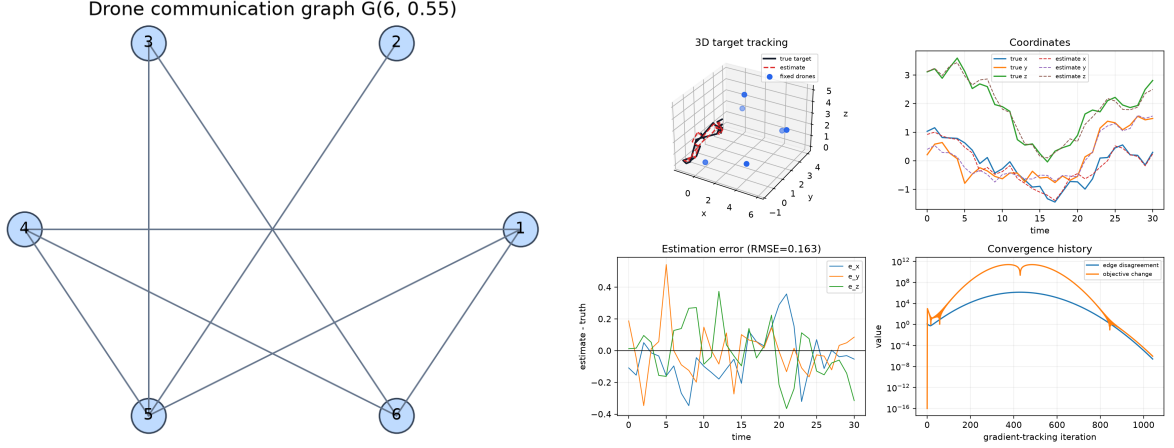


Figure 3: Drone graph and the true/estimated target path, coordinate errors, and consensus check.

2.3 Numerical settings and result

$$\bar{z}_0 = (2, -1, 3)^\top, \quad \Sigma_0 = \text{diag}(0.8, 0.8, 0.6), \quad \Sigma_w = \text{diag}(0.12, 0.10, 0.16).$$

The fixed drone locations are

$$\begin{bmatrix} 0 & 0 & 0 \\ 4 & 0 & 1 \\ 0 & 4 & 2 \\ 4 & 4 & 1 \\ 2 & 2 & 5 \\ 6 & 2 & 3 \end{bmatrix}.$$

The sensor covariance for drone i is $(1 + 0.1i) \text{diag}(0.22, 0.20, 0.28)$ with $i = 0, \dots, 5$. The corresponding mixing matrix is

$$W = \begin{bmatrix} .30 & 0 & 0 & .25 & .20 & .25 \\ 0 & .80 & 0 & 0 & .20 & 0 \\ 0 & 0 & .55 & 0 & .20 & .25 \\ .25 & 0 & 0 & .30 & .20 & .25 \\ .20 & .20 & .20 & .20 & .20 & 0 \\ .25 & 0 & .25 & .25 & 0 & .25 \end{bmatrix},$$

whose row and column sums are one. The final run used 1043 iterations. The final edge disagreement was 2.64×10^{-7} and the trajectory RMSE was 0.162559. The estimate begins at (0.9233, 0.4000, 3.1256) and ends at (0.2448, 1.5676, 2.4973).

The estimate follows the true trajectory closely in all three coordinates. The error plot provides the clearest check: its three curves stay close to zero, and the RMSE is small compared with the target's motion. The convergence panel records edge disagreement, while the stopping rule also checks objective change. The graph and sensor locations remain fixed throughout the simulation.

Problem 2 code

The exact source file is also available at [GitHub: Assignment_3_Problem_2.py](#).

```
"""Problem 2: distributed gradient-tracking estimate of a moving target."""
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent; ASSET = ROOT / "assets"; ASSET.mkdir(exist_ok=True)
SEED, N, P, TMAX = 20260905, 6, 0.55, 30
ALPHA, MAX_ITERS = 0.018, 7000

def graph_and_weights():
    for s in range(SEED, SEED + 100):
        rng = np.random.default_rng(s); upper = rng.random((N, N)) < P
        A = np.triu(upper, 1).astype(float); A += A.T
        seen = {0}; frontier = [0]
        while frontier:
            i = frontier.pop()
            for j in np.flatnonzero(A[i]):
                if int(j) not in seen: seen.add(int(j)); frontier.append(int(j))
            if len(seen) == N: break
        else: raise RuntimeError("No connected graph found.")
        deg = A.sum(axis=1).astype(int); W = np.zeros((N, N))
        edges = list(zip(*np.where(np.triu(A, 1) > 0)))
        for i, j in edges: W[i, j] = W[j, i] = 1 / (1 + max(deg[i], deg[j]))
        for i in range(N): W[i, i] = 1 - W[i].sum()
    return A, W, s

def grad(local, q, mean, inv0, invw, invv):
    g = np.zeros_like(local)
    g[:, 0] += (2 / N) * ((local[:, 0] - mean) @ inv0.T)
    g[:, 1:] += (2 / N) * ((local[:, 1:] - local[:, :-1]) @ invw.T)
    g[:, :-1] -= (2 / N) * ((local[:, 1:] - local[:, :-1]) @ invw.T)
    g += 2 * (local - q) @ invv.transpose(0, 2, 1)
    return g

def objective(local, q, mean, inv0, invw, invv):
    total = np.zeros(N)
    for i in range(N):
        val = np.sum((local[i, 0] - mean) @ inv0 * (local[i, 0] - mean)) / N
        d = local[i, 1:] - local[i, :-1]
        val += np.sum((d @ invw) * d) / N
        e = q[i] - local[i]
        val += np.sum((e @ invv[i]) * e)
        total[i] = val
    return total

def main():
    rng = np.random.default_rng(SEED); A, W, graph_seed = graph_and_weights(); edges = list(zip(*np.where(
        np.triu(A, 1) > 0)))
    mean = np.array([2.0, -1.0, 3.0]); S0 = np.diag([0.8, 0.8, 0.6])
    Sw = np.diag([0.12, 0.10, 0.16]); Sv = np.array([np.diag([0.22, 0.20, 0.28]) * (1 + .1*i) for i in
        range(N)])
    inv0, invw, invv = np.linalg.inv(S0), np.linalg.inv(Sw), np.linalg.inv(Sv)
    z = np.zeros((TMAX + 1, 3)); z[0] = rng.multivariate_normal(mean, S0)
    for t in range(TMAX): z[t + 1] = z[t] + rng.multivariate_normal(np.zeros(3), Sw)
    x = np.array([[0, 0, 0], [4, 0, 1], [0, 4, 2], [4, 4, 1], [2, 2, 5], [6, 2, 3]], dtype=float)
    v = np.array([rng.multivariate_normal(np.zeros(3), Sv[i // (TMAX + 1)]) for i in range(N * (TMAX + 1))
        ]).reshape(N, TMAX + 1, 3)
    y = x[:, None, :] - z[None, :, :] + v
    q = x[:, None, :] - y

    local = np.tile(mean, (N, TMAX + 1, 1)); local += rng.normal(0, .05, local.shape)
    tracker = grad(local, q, mean, inv0, invw, invv); history = []; previous_obj = None
```

```

for k in range(MAX_ITERS):
    a = ALPHA / (1 + 0.0004 * k)
    nxt = np.einsum("ij,jtd->itd", W, local) - a * tracker
    tracker = np.einsum("ij,jtd->itd", W, tracker) + grad(nxt, q, mean, inv0, invw, invv) - grad(local
, q, mean, inv0, invw, invv)
    local = nxt
    disagreement = max((np.linalg.norm(local[i] - local[j]) for i, j in edges), default=0.0)
    obj = float(objective(local, q, mean, inv0, invw, invv).sum())
    history.append((disagreement, obj))
    if k > 100 and disagreement < 2e-4 and previous_obj is not None and abs(obj - previous_obj) < 1e
-6: break
    previous_obj = obj
estimate = local.mean(axis=0); err = estimate - z; rmse = float(np.sqrt(np.mean(err ** 2)))

fig = plt.figure(figsize=(10, 7)); ax = fig.add_subplot(221, projection="3d")
ax.plot(z[:, 0], z[:, 1], z[:, 2], color="#111827", lw=2, label="true target")
ax.plot(estimate[:, 0], estimate[:, 1], estimate[:, 2], color="#dc2626", ls="--", label="estimate")
ax.scatter(x[:, 0], x[:, 1], x[:, 2], color="#2563eb", s=35, label="fixed drones")
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z"); ax.set_title("3D target tracking"); ax.
legend(fontsize=7)
ax2 = fig.add_subplot(222); ax2.plot(z, lw=1.5, label=["true x", "true y", "true z"]); ax2.plot(
estimate, ls="--", lw=1.0, label=["estimate x", "estimate y", "estimate z"]); ax2.set_title("
Coordinates"); ax2.set_xlabel("time"); ax2.grid(alpha=.2); ax2.legend(fontsize=7, ncol=2)
ax3 = fig.add_subplot(223); ax3.plot(err, lw=1.0, label=["e_x", "e_y", "e_z"]); ax3.axhline(0, color="
black", lw=.8); ax3.set_title(f"Estimation error (RMSE={rmse:.3f})"); ax3.set_xlabel("time"); ax3.
set_ylabel("estimate - truth"); ax3.grid(alpha=.2); ax3.legend(fontsize=8)
ax4 = fig.add_subplot(224); h = np.asarray(history); objective_change = np.abs(np.diff(h[:, 1],
prepend=h[0, 1])); objective_change[0] = max(objective_change[0], 1e-16); ax4.semilogy(np.maximum(h
[:, 0], 1e-16), label="edge disagreement"); ax4.semilogy(np.maximum(objective_change, 1e-16), label="
objective change"); ax4.set_title("Convergence history"); ax4.set_xlabel("gradient-tracking iteration
"); ax4.set_ylabel("value"); ax4.grid(alpha=.2); ax4.legend(fontsize=7)
fig.tight_layout(); fig.savefig(ASSET / "p2_tracking_results.png", dpi=220); plt.close(fig)

fig, ax = plt.subplots(figsize=(5.2, 4.2)); theta = np.linspace(0, 2*np.pi, N, endpoint=False); pos =
np.c_[np.cos(theta), np.sin(theta)]
for i,j in edges: ax.plot([pos[i,0],pos[j,0]],[pos[i,1],pos[j,1]],color="#64748b",lw=1)
ax.scatter(pos[:,0],pos[:,1],s=500,c="#bfdbfe",edgecolors="#334155")
for i in range(N): ax.text(pos[i,0],pos[i,1],str(i+1),ha="center",va="center")
ax.set_title(f"Drone communication graph G({N}, {P})"); ax.axis("off"); fig.tight_layout(); fig.
savefig(ASSET / "p2_graph.png", dpi=220); plt.close(fig)
print(f"seed={SEED}, graph_seed={graph_seed}, N={N}, p={P}, TMAX={TMAX}, iterations={len(history)}")
print(f"connected=True, alpha0={ALPHA}, final_disagreement={history[-1][0]:.6e}, RMSE={rmse:.6f}")
print("W="); print(W)
print("drone locations="); print(x)
print("final estimate first/last="); print(estimate[0], estimate[-1])

if __name__ == "__main__": main()

```

Problem 3

There are $N \in \mathbb{N}$ agents connected via an undirected connected graph $G(V, E)$. They must collaboratively execute $M \in \mathbb{N}$ tasks where $M > N$. The set of all tasks is denoted by $\mathcal{T} := \{T_1, \dots, T_M\}$. Each agent $i \in V$ is assigned a local cost C_{ij} for performing a task $T_j \in \mathcal{T}$ and thereby giving us a cost matrix $C \in \mathbb{R}^{N \times M}$. Further, each agent $i \in V$ has a capacity constraint $b_i \in \mathbb{Z}_{\geq 0}$ which captures how many tasks that agent can take. Since there are totally M tasks and each task should be assigned only once, $\sum_{i=1}^N b_i = M$. Do the following:

1. Formulate the task allocation problem as an optimization problem with appropriate cost function and constraints. Your understanding of the problem and the way you formulate should be highlighted well. Give the ADMM updates for every agent $i \in V$. The result should be a binary linear program, which is NP-hard to solve for large M, N . Hence, you will need to do convex relaxation.
2. Randomly choose the number of tasks M and number of agents N such that $M > N$. Fix them and generate an undirected connected graph $G(V, E)$ for the communication model (use Erdos-Renyi model). Randomly choose and fix the capacities b_i for every agent such that $\sum_i b_i = M$. Generate a random cost matrix $C \in \mathbb{R}^{N \times M}$ and ensure each element $C_{ij} > 0$. Plot the graph $G(V, E)$ and the result of task allocation using ADMM simulation. Fix the same (M, N) , play with different generated cost matrices C and plot the resulting task allocation and explain your observations.

Solution: I choose $N = 4$ agents and $M = 7$ tasks, so $M > N$. The capacities are $b = (2, 2, 2, 1)$ and sum to 7. The connected graph uses $p = 0.70$ and graph seed 20260906; its edges are $(1, 2), (2, 3), (2, 4), (3, 4)$. The three positive cost matrices use the same dimensions and are generated successively from the fixed NumPy seed 20260905.

Complexity note. With only the row-capacity, one-owner-per-task, and binary constraints shown in the question, this is a capacitated transportation (min-cost-flow) problem. Its integral row and column totals give an integral LP relaxation, so it is not generally NP-hard in this exact form. I solve the stated model without adding a rounding step. The NP-hard hint in the question would require additional constraints or a different allocation model.

3.1 Integer model and convex relaxation

Let $x_{ij} = 1$ if agent i performs task T_j , and $x_{ij} = 0$ otherwise. With $X = [x_{ij}] \in \{0, 1\}^{N \times M}$, the integer problem is

$$\begin{aligned}
\min_X \quad & \langle C, X \rangle = \sum_{i=1}^N \sum_{j=1}^M C_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{i=1}^N x_{ij} = 1, \quad j = 1, \dots, M, \\
& \sum_{j=1}^M x_{ij} = b_i, \quad i = 1, \dots, N, \\
& x_{ij} \in \{0, 1\}.
\end{aligned}$$

The column constraint gives every task exactly one owner. The row constraint limits each agent to its capacity, while the binary constraint prevents a task from being split between agents.

I relax the last constraint to $0 \leq x_{ij} \leq 1$. Because this is a transportation problem with integer row and column totals, an optimum of the relaxed problem is integral. I still check this numerically before displaying the final binary matrix.

3.2 ADMM splitting and updates

For the numerical reference implementation, agent i keeps the row $x_i \in \mathbb{R}^M$ and a coordination copy $Z_{i,:} \in \mathbb{R}^M$. The matrix Z is maintained by a coordinator in this implementation; therefore the updates below are consensus-constrained ADMM updates, not a claim of peer-to-peer ADMM over the plotted graph. Define

$$\mathcal{X}_i = \{x_i \in [0, 1]^M : \mathbf{1}^\top x_i = b_i\}, \quad \mathcal{Z} = \{Z \in [0, 1]^{N \times M} : \mathbf{1}^\top Z_{:,j} = 1 \ \forall j\}.$$

The relaxed problem is

$$\min_{X, Z} \sum_{i=1}^N \left(C_i^\top x_i + I_{\mathcal{X}_i}(x_i) \right) + I_{\mathcal{Z}}(Z) \quad \text{subject to } X - Z = 0.$$

Here $I_{\mathcal{S}}$ is zero on $\mathcal{S}$ and infinite outside it. With scaled dual matrix $U \in \mathbb{R}^{N \times M}$ and penalty $\rho > 0$, the updates are

$$\begin{aligned} x_i^{k+1} &= \arg \min_{x_i \in \mathcal{X}_i} \left\{ C_i^\top x_i + \frac{\rho}{2} \|x_i - z_i^k + u_i^k\|_2^2 \right\}, \\ Z^{k+1} &= \arg \min_{Z \in \mathcal{Z}} \frac{\rho}{2} \|X^{k+1} - Z + U^k\|_F^2, \\ U^{k+1} &= U^k + X^{k+1} - Z^{k+1}. \end{aligned}$$

The first step is a local row problem. The second step adjusts each task column so that its entries are nonnegative and sum to one; this is how the agents agree on task ownership. The graph is used for the communication setting and connectivity check, but the central Z projection is not a peer-to-peer message-passing step.

For completeness, a genuine peer-to-peer version would give every node i a full local copy $X_i^{\text{loc}} \in \mathbb{R}^{N \times M}$ and use

$$\mathcal{C}_i = \left\{ Y \in [0, 1]^{N \times M} : \mathbf{1}^\top Y_{:,j} = 1 \ \forall j, \quad \mathbf{1}^\top Y_{i,:} = b_i \right\}.$$

For each undirected edge (i, j) it would introduce an edge copy Y_{ij} and impose the two consensus constraints $X_i^{\text{loc}} = Y_{ij}$ and $X_j^{\text{loc}} = Y_{ij}$. With oriented scaled duals U_{ij} and U_{ji} , the edge-wise updates are

$$\begin{aligned} X_i^{k+1} &= \arg \min_{Y \in \mathcal{C}_i} \left\{ C_{i,:} Y_{i,:}^\top + \frac{\rho}{2} \sum_{j \in \mathcal{N}_i} \|Y - Y_{ij}^k + U_{ij}^k\|_F^2 \right\}, \\ Y_{ij}^{k+1} &= \frac{1}{2} \left(X_i^{k+1} + U_{ij}^k + X_j^{k+1} + U_{ji}^k \right), \\ U_{ij}^{k+1} &= U_{ij}^k + X_i^{k+1} - Y_{ij}^{k+1}. \end{aligned}$$

The last dual update is run at both ends of every edge. Node i sends X_i^{k+1} only to its neighbours, and each edge forms its average from the two received copies. This is the proper peer-to-peer formulation, but the figures below use the simpler coordinator-based reference solver.

Both projections use the same simple rule. For a vector v , the projection onto $\{q : 0 \leq q \leq 1, \mathbf{1}^\top q = b\}$ is

$$\Pi_b(v) = \text{clip}(v - \theta, 0, 1),$$

where the scalar θ is chosen so that the entries sum to b . Thus the row update applies this rule to $z_i^k - u_i^k - C_i/\rho$, and the column update applies it to $X_{:j}^{k+1} + U_{:j}^k$ with $b = 1$.

3.3 Simulation and results

I use $\rho = 1$, primal and dual tolerances 10^{-5} , and at most 12000 iterations. The ADMM residuals are $r^k = X^k - Z^k$ and $s^k = \rho(Z^k - Z^{k-1})$. All three runs reached both tolerances, and all relaxed solutions were integral up to 2×10^{-4} ; no rounding or repair was needed.

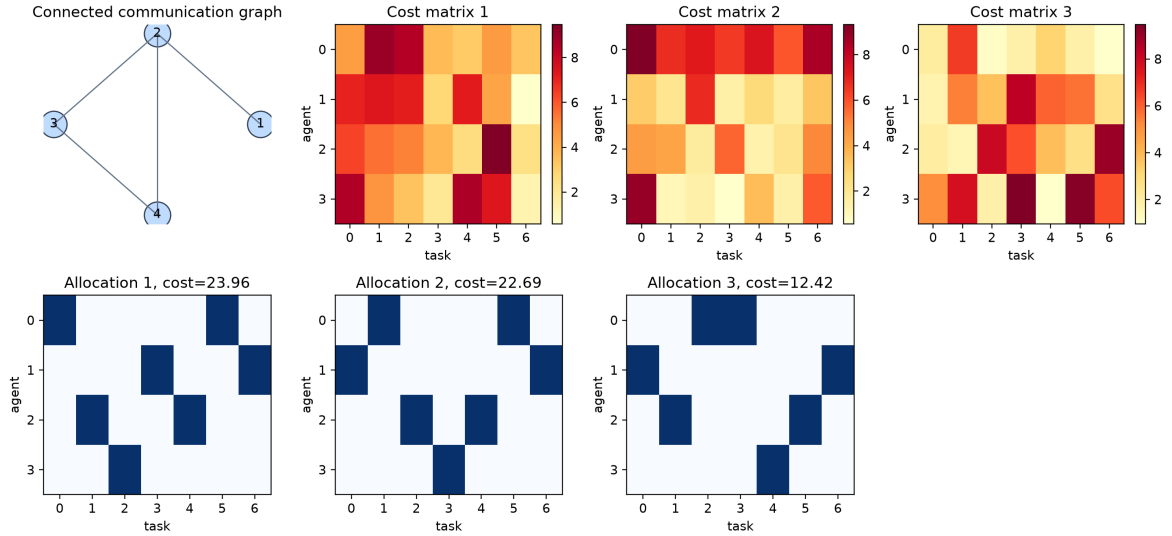


Figure 4: Communication graph, positive cost heatmaps, and the three binary task allocations.

case	iters	$\ r\ $	$\ s\ $	total cost	row	column	changed
baseline	9	5.44×10^{-16}	0	23.962868	pass	pass	0
case 2	11	5.21×10^{-16}	4.97×10^{-16}	22.693173	pass	pass	4
case 3	6	5.33×10^{-16}	2.54×10^{-16}	12.415220	pass	pass	5

The three displayed binary allocation matrices are

$$X^{(1)} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \quad X^{(2)} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \quad X^{(3)} = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

Each matrix has column sums equal to one and row sums equal to $(2, 2, 2, 1)$.

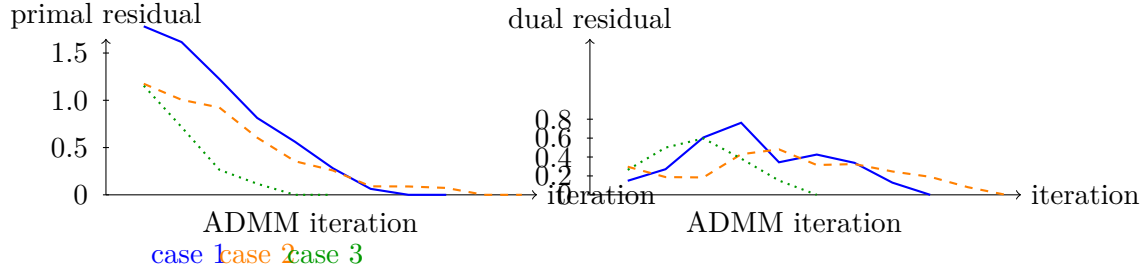


Figure 5: Primal and dual residual histories for the three fixed cost matrices.

The task-owner rows below list the owner of tasks T_1 through T_7 :

baseline	1	3	4	2	3	1	2
case 2	2	1	3	4	3	1	2
case 3	2	3	1	1	4	3	2

The baseline assignment gives two tasks to agents 1, 2, and 3, and one to agent 4. Every column has one owner and the row counts are (2, 2, 2, 1). With a new cost matrix, task preferences change, so the capacity limits can move tasks even when an agent has a low cost for them. The changes are therefore explained by the new costs and capacity limits, not by random behaviour.

Problem 3 code

The exact source file is also available at [GitHub: Assignment_3_Problem_3.py](#).

```

"""Problem 3: consensus-constrained transportation ADMM reference solver."""
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent; ASSET = ROOT / "assets"; ASSET.mkdir(exist_ok=True)
SEED, N, M, P, RHO = 20260905, 4, 7, 0.70, 1.0
CAPACITY = np.array([2, 2, 2, 1], dtype=float); TOL = 1e-5; MAX_ITERS = 12000

def graph():
    for s in range(SEED, SEED + 100):
        rng = np.random.default_rng(s); upper = rng.random((N, N)) < P
        A = np.triu(upper, 1).astype(float); A += A.T
        seen = {0}; frontier = [0]
        while frontier:
            i = frontier.pop()
            for j in np.flatnonzero(A[i]):
                if int(j) not in seen: seen.add(int(j)); frontier.append(int(j))
            if len(seen) == N: return A, s
        raise RuntimeError("No connected graph found.")

def project_capped_simplex(v, total):
    lo, hi = float(np.min(v) - 1), float(np.max(v) + 1)
    for _ in range(80):
        mid = (lo + hi) / 2
        if np.clip(v - mid, 0, 1).sum() > total: lo = mid
        else: hi = mid
    return np.clip(v - (lo + hi) / 2, 0, 1)

def solve(C):
    X = np.vstack([project_capped_simplex(-C[i] / RHO, CAPACITY[i]) for i in range(N)])

```

```

Z = np.vstack([project_capped_simplex(X[:, j], 1) for j in range(M)]).T
U = np.zeros((N, M)); previous_Z = Z.copy(); history = []
for k in range(1, MAX_ITERS + 1):
    X = np.vstack([project_capped_simplex(Z[i] - U[i] - C[i] / RHO, CAPACITY[i]) for i in range(N)])
    previous_Z = Z.copy(); Z = np.vstack([project_capped_simplex(X[:, j] + U[:, j], 1) for j in range(
M)]).T
    U += X - Z
    primal = np.linalg.norm(X - Z); dual = RHO * np.linalg.norm(Z - previous_Z)
    history.append((primal, dual))
    if primal < TOL and dual < TOL: break
binary = (Z > 0.5).astype(int)
# The transportation relaxation is integral here; this check is the only post-processing.
integral = bool(np.max(np.minimum(np.abs(Z), np.abs(Z - 1))) < 2e-4)
cost = float(np.sum(C * binary))
feasible = bool(np.all(binary.sum(axis=0) == 1) and np.all(binary.sum(axis=1) == CAPACITY))
return Z, binary, k, primal, dual, integral, feasible, cost, history

def main():
    rng = np.random.default_rng(SEED); A, graph_seed = graph(); edges = list(zip(*np.where(np.triu(A, 1) >
0)))
    costs = [rng.uniform(0.5, 9.5, (N, M)) for _ in range(3)]
    results = [solve(C) for C in costs]
    owners = [r[1].argmax(axis=0) for r in results]
    changed = [0] + [int(np.count_nonzero(owners[i] != owners[0])) for i in range(1, 3)]

    fig, axes = plt.subplots(2, 4, figsize=(13.2, 6.2))
    theta = np.linspace(0, 2*np.pi, N, endpoint=False); pos = np.c_[np.cos(theta), np.sin(theta)]
    for i, j in edges: axes[0,0].plot([pos[i,0], pos[j,0]], [pos[i,1], pos[j,1]], color="#64748b", lw=1)
    axes[0,0].scatter(pos[:,0], pos[:,1], s=450, c="#bfdbfe", edgecolors="#334155")
    for i in range(N): axes[0,0].text(pos[i,0], pos[i,1], str(i+1), ha="center", va="center")
    axes[0, 0].set_title("Connected communication graph"); axes[0, 0].axis("off")
    for j, (C, result) in enumerate(zip(costs, results)):
        im = axes[0, j + 1].imshow(C, cmap="YlOrRd", aspect="auto"); axes[0, j + 1].set_title(f"Cost
matrix {j+1}"); axes[0, j + 1].set_xlabel("task"); axes[0, j + 1].set_ylabel("agent")
        axes[0, j + 1].set_xticks(range(M)); axes[0, j + 1].set_yticks(range(N)); fig.colorbar(im, ax=axes
[0, j + 1], fraction=.046)
        axes[1, j].imshow(result[1], cmap="Blues", vmin=0, vmax=1, aspect="auto"); axes[1, j].set_title(f"
Allocation {j+1}, cost={result[7]:.2f}"); axes[1, j].set_xlabel("task"); axes[1, j].set_ylabel("agent
"); axes[1, j].set_xticks(range(M)); axes[1, j].set_yticks(range(N))
    axes[1, 3].axis("off")
    fig.tight_layout(); fig.savefig(ASSET / "p3_allocation_results.png", dpi=220); plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.4))
    for idx, result in enumerate(results, 1):
        hist = np.asarray(result[8])
        axes[0].semilogy(np.maximum(hist[:, 0], 1e-16), label=f"case {idx}")
        axes[1].semilogy(np.maximum(hist[:, 1], 1e-16), label=f"case {idx}")
    axes[0].set_title("Primal residual"); axes[1].set_title("Dual residual")
    for ax in axes:
        ax.set_xlabel("ADMM iteration"); ax.set_ylabel("residual norm"); ax.grid(alpha=.2); ax.legend(
fontsize=8)
    fig.tight_layout(); fig.savefig(ASSET / "p3_residuals.png", dpi=220); plt.close(fig)

    print(f"seed={SEED}, graph_seed={graph_seed}, N={N}, M={M}, p={P}, rho={RHO}, capacities={CAPACITY.
astype(int).tolist()}")
    print(f"connected=True, edges=[[i+1,j+1] for i,j in edges]]")
    for idx, (C, result) in enumerate(zip(costs, results), 1):
        Z, binary, iters, primal, dual, integral, feasible, cost, history = result
        print(f"case {idx}: iterations={iters}, primal={primal:.3e}, dual={dual:.3e}, integral={integral},
feasible={feasible}, cost={cost:.6f}, changed={changed[idx-1]}")
        print("row sums=", binary.sum(axis=1), "column sums=", binary.sum(axis=0))
        print("C="); print(np.round(C, 4)); print("X="); print(binary)

if __name__ == "__main__": main()

```

Problem 4

Consider the LASSO problem

$$\min_x \left[\frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1 \right].$$

Write the ADMM iteration updates by solving and writing the closed form solution for x, z updates.

Solution: Assume $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $x, z \in \mathbb{R}^n$, $\lambda \geq 0$, and $\rho > 0$. Introduce z and impose $x - z = 0$:

$$\min_{x, z} \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|z\|_1 \quad \text{subject to } x - z = 0.$$

This split keeps the least-squares term with x and the absolute-value penalty with z .

Using the scaled dual variable u and penalty $\rho > 0$, the scaled augmented Lagrangian is

$$\mathcal{L}_\rho(x, z, u) = \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|z\|_1 + \frac{\rho}{2} \|x - z + u\|_2^2 - \frac{\rho}{2} \|u\|_2^2.$$

4.1 The x update

Holding z^k and u^k fixed, minimise the smooth part. Its gradient is

$$A^\top (Ax - b) + \rho(x - z^k + u^k).$$

Setting it to zero gives the normal equation

$$(A^\top A + \rho I)x^{k+1} = A^\top b + \rho(z^k - u^k).$$

Since $A^\top A$ is positive semidefinite and ρI is positive definite, $A^\top A + \rho I$ is invertible. Hence the closed form is

$$x^{k+1} = (A^\top A + \rho I)^{-1} (A^\top b + \rho(z^k - u^k)).$$

In implementation, solve this linear system directly instead of computing the matrix inverse.

4.2 The z update

Set $v = x^{k+1} + u^k$. The z subproblem is

$$z^{k+1} = \arg \min_z \left\{ \lambda \|z\|_1 + \frac{\rho}{2} \|z - v\|_2^2 \right\}.$$

It separates over coordinates. For one coordinate, minimise $\lambda|z_j| + \frac{\rho}{2}(z_j - v_j)^2$. The three cases are:

$$z_j = \begin{cases} v_j - \lambda/\rho, & v_j > \lambda/\rho, \\ 0, & |v_j| \leq \lambda/\rho, \\ v_j + \lambda/\rho, & v_j < -\lambda/\rho. \end{cases}$$

Therefore the closed form is the soft-threshold operation

$$z^{k+1} = S_{\lambda/\rho}(x^{k+1} + u^k), \quad [S_\kappa(v)]_j = \text{sign}(v_j) \max\{|v_j| - \kappa, 0\}.$$

In simple terms: small entries become zero, while larger entries move toward zero.

4.3 Dual update and complete iteration

The scaled dual update is

$$u^{k+1} = u^k + x^{k+1} - z^{k+1}.$$

The quantity $x^{k+1} - z^{k+1}$ is the primal feasibility residual. Thus one complete ADMM iteration is the linear solve for x , soft thresholding for z , and the dual update above.

AI-use declaration

I used AI assistance only to improve the final writing of this assignment.